

Assignment 1 - Detailed Feedback Report

Student ID: 252608

Group code: ali-ahm1

Repository: <https://github.com/AliTrabeh/sine-frequency-classifier>

Your grade before bonus 64.23 and the final grade after bonus is 74.23

Important Disclaimer and Appeal Policy

This report was prepared with the assistance of an AI agent; therefore, errors may occur. The evaluation is subjective and qualitative rather than quantitative. The purpose of these comments is to help understand where and how improvements can be made. These notes do not serve as a benchmark for comparison between different assignments or grades; each evaluation is strictly individual.

To compensate for potential discrepancies and reading errors, the system automatically awards a 10-point bonus. Consequently, if you are considering an appeal, there is generally no need to do so, as the bonus already accounts for this.

Should you choose to submit a formal appeal regardless, please note the following conditions:

1. The 10-point bonus will be canceled.
2. The previously assigned grade will be nullified.
3. A comprehensive, thorough re-evaluation of the entire assignment will be conducted.
4. The final grade will be recalculated from scratch without the bonus, and without consideration of the pre-appeal score.

Assessment Overview

Your submission demonstrates a foundation that shows potential but requires substantial development. While fundamental elements are in place, multiple areas require focused attention. The recommendations below outline a clear path forward.

A note regarding your self-assessment: Your submission was reviewed with particularly high attention to detail and precision. When a student expresses strong confidence in the quality of their work, the evaluation is conducted with an especially rigorous lens — ensuring the assessment fully reflects professional engineering standards across every dimension. This thorough level of scrutiny is part of what shaped the outcome of your assessment.

Areas of Excellence

Your submission demonstrates particular strength in the following areas:

Project Planning

Your project planning demonstrates structured thinking and clear articulation of both the problem space and the technical approach. The documentation would allow a new team member to understand your vision and design decisions independently.

Code Documentation

Your documentation clearly communicates the purpose, setup, and usage of your project, reflecting professional standards that would allow any developer to get started without prior knowledge of your work.

Testing Quality

Your testing demonstrates a rigorous approach to validating your implementation across a meaningful range of scenarios, including edge cases. This reflects strong commitment to code reliability.

Research & Analysis

Your analytical work clearly documents your experimental process and the insights you derived from it, providing strong evidence of genuine understanding rather than just a final result.

Version Management

Your version control and development process documentation reflects disciplined, professional engineering habits. The evolution of your project — including your AI-assisted

workflow — is clearly visible.

Mixed-Signal Generation

Your construction of the mixed input signal is principled — the constituent components and their combination are handled in a way that gives the network a clean, well-defined learning problem to solve.

Areas for Improvement and Development

The following areas require focused attention and development. Addressing these will significantly strengthen your engineering practices and overall project quality:

Configuration & Security

Your configuration management is mostly solid, but there are aspects of how your project handles its environment that could be more professional and portable. Consider how your project would be set up by someone in a completely different environment who has never seen your configuration before.

UI/UX

The user-facing dimension of your project is not fully communicated. How someone experiences and interacts with your system — and how you present that experience — should be a considered part of your submission. Think about how you would help a reader understand what it looks and feels like to actually use your work, without having to run it themselves.

Costs & Pricing

There is no evidence of cost or resource awareness in your project. Any real-world AI application must consider the economic implications of its design choices. A professional would document and reason about what their solution costs to operate and how those costs behave as usage scales.

Extensibility

Your project's design could better support future growth and modification. Professional software is built with the expectation that requirements will change. Think about how clearly your architecture separates concerns, and how easily a new developer could extend your system without disrupting what already works.

Quality Standards

Your project's approach to maintaining code quality is not clearly established. Modern professional development relies on automated tooling and processes to ensure consistency and catch issues early. Think about what mechanisms in your project enforce quality standards beyond manual review.

Per-Sample Noise Variation

The noise in your dataset does not change meaningfully across consecutive samples. This is a foundational issue for time-series learning: noise that is constant within a signal gives the network nothing to denoise — it just looks like a slightly different clean signal. Professional ML data pipelines re-draw stochastic perturbations at the sample level so the network must learn statistical regularities, not memorise specific realisations.

Clean Target Generation

Your target signals do not clearly separate the pure underlying structure from the noise. The whole point of a denoising-style task is to give the model an unambiguous clean reference. Think about what the network is supposed to output: any contamination in the target turns the learning problem into something fundamentally different.

Train/Test Seed Split

Your train and test sets do not reflect the discipline a generalisation test requires. The two splits must share the same structural definition but differ in their stochastic realisations — otherwise you are not measuring generalisation, you are measuring something else. Review how your data construction handles the boundary between the two splits.

Dataset Shape

The shape and scale of your dataset, or the dimensionality of your network's input, is not matched to what the task requires. Think about what each input feature represents and how many samples the network needs to see to actually learn the underlying signal — both quantities follow from the structure of the problem.

One-Hot Input Format

The way your model receives its conditioning signal — which target it should produce on a given example — is not as clean as it could be. Professional practice is to make the conditioning input crisp and discrete, leaving no ambiguity about what the network is being asked to do on each call.

LSTM Model Architecture

Your model architecture does not fully exploit the recurrent structure the task is asking for. The whole reason to use an LSTM is that it can carry information through time — if the architecture doesn't actually use that capability, or if the input/output dimensions are mismatched with the data, the design works against the model rather than with it.

Internal State Preservation

Your training does not preserve the network's internal recurrent state in the way the task demands. The temporal advantage of an LSTM comes from carrying memory across consecutive steps; if that memory is reset at every step, the network is operating as a stateless model with extra parameters. Review where in your training loop state is being reset and whether that reset is intentional.

Sequence-Length Justification

Your choice of sequence length is not adequately justified for the chosen design. Whether you process samples one at a time or in windows of several is a substantive design decision with real consequences for what the model can learn. The justification for the choice — and the trade-offs it implies — should be visible to a reader.

Training Loop Correctness

Your training procedure does not clearly demonstrate that learning is happening. A professional training implementation tracks losses across epochs, ensures the optimiser updates the parameters correctly, and produces evidence that the loss actually decreases. Without those elements visible, it is hard to know whether the model is learning anything.

Training Set Evaluation

Your reporting of the model's performance on the training set is incomplete or ambiguous. Even when training looks successful, the formal training-set metric must be computed cleanly and reported clearly — it is the baseline against which every other claim about the model is measured.

Test Set Evaluation

Your evaluation on held-out data is incomplete or ambiguous. A model that performs well on data it has seen says nothing about its generalisation; the test-set measurement on data drawn from a separate stochastic realisation is what tells you whether the model actually learned something.

Generalization Analysis

Your analysis stops at reporting numbers — it does not interpret what they mean. A professional discussion compares train and test metrics, names the qualitative outcome (good fit, overfitting, underfitting), and reasons about what the gap implies for the model's behaviour. Numbers alone are not analysis.

Single-Frequency Visualization

Your visualisation does not communicate the model's behaviour clearly. A reader should be able to see, at a glance, what the network was given as input, what it was supposed to produce, and what it actually produced — layered together so the relationship is obvious. Without that, the result is hard to interpret.

All-Frequencies Visualization

Your visualisation of the full set of frequencies does not communicate how the model's behaviour varies across them. A professional analysis shows each target separately so a

reader can compare the model's performance across conditions, rather than collapsing everything into a single overloaded chart.

Conclusion

Focus first on the most critical gaps and then address each area in turn. Use this feedback as a roadmap for improvement and continued growth as a software engineer.

Assessment Date: 2026-05-18

This report provides developmental feedback to support your learning and growth.